IEEE 802.11AI-2016 • IEEE 802.11-2024 §12.11

FILS 认证协议流程图

Fast Initial Link Setup — 两帧认证 + 两帧关联，完成双向认证、PMK/PTK 建立与密钥确认

预共享知识 / 前提　发送前计算
消息字段　接收后处理
密钥 / 结果　失败 / 终止

1 协议实体与信任模型

PK 路径无 TTP；SK 路径的 TTP 交互发生在 AP 与 TTP 之间

STA

发起方 / 被认证方

non-AP Station

- **PK：**持有自身私钥，并可信 AP 的公钥/证书链
- **SK：**与 TTP 共享 `rRK`，或持有缓存的 `PMKSA`
- 生成 `SNonce`、临时 DH 密钥、FILS Session
- 推导 PMK/PTK，发送 KeyAuth，验证 AP 的 KeyAuth

AP

响应方 / Authenticator

FILS-capable AP

- **PK：**持有自身私钥，并可信 STA 的公钥/证书链
- **SK：**可选缓存 `PMKSA`；可连接 TTP
- 生成 `ANonce`、临时 DH 密钥，校验对端公钥
- 转发 EAP-RP，验证 KeyAuth，经 Key Delivery 下发 GTK

TTP / AS

仅 SK 路径

Trusted Third Party / Authentication Server

- 与 STA 预先通过完整 EAP 建立 `rRK`
- 验证 `EAP-Initiate/Reauth`
- 返回 `EAP-Finish/Reauth` 与 `rMSK`
- AP↔TTP 使用 RADIUS/Diameter，不在 802.11 范围内



2 实体间共享知识

“本次建立”一列是认证帧交换真正新增的秘密材料

路径	STA 预共享	AP 预共享	TTP 预共享	本次交换建立
FILS Public Key 算法 6	自身私钥 + 可信 AP 公钥/证书链	自身私钥 + 可信 STA 公钥/证书链	—	`gSTA, gAP → DHss` 双向签名认证
FILS Shared Key 算法 5（with PFS）	`rRK` + 支持的有限循环群	有限循环群；可选缓存 `PMKSA`	`rRK`	`rMSK + DHss` 前向保密
PMKSA 缓存 SK / PK 均可	缓存 `PMK + PMKID`	缓存 `PMK + PMKID`	不参与（命中时）	不新建 PMKSA； 仅新建 `PTKSA`

有限循环群：从 `dot11RSNAConfigDLCGroupTable` 选择双方共同支持的群　**哈希：**由 AKM 决定 SHA-256 / SHA-384　**Nonce：**SNonce / ANonce 各 16 octets　**会话绑定：**FILS Session 标识本次交换　**帧保护：**AEAD（AES-SIV-256/512），密钥为 PTK-KEK

3 四帧总览：两帧认证 + 两帧关联

SK 的 AP↔TTP 交互不增加 STA 的往返次数

仅 SK 路径： AP ⇄ TTP（EAP-RP over RADIUS/Diameter）—— 与 STA 的第 1、2 帧并行发生，不占 STA 空中往返 ｜ **PK 路径：** 无此交互



① + ②：密钥建立 —— 产生 PMK / PMKID / PTK

③ + ④：密钥确认 —— KeyAuth + AEAD，无 4 次 EAPOL 握手

两条路径速查对照

同一四帧框架，不同信任根与密钥材料

维度	FILS Public Key（算法 6）	FILS Shared Key（算法 4 / 5）
信任根	X.509v3 证书链，或带指纹的可信公钥；无 TTP	STA 与 TTP 共享的 `rRK`；AP 经 TTP 取得 `rMSK`
TTP 参与	无	有（仅 EAP-RP 路径；命中 PMKSA 缓存时跳过）
临时 DH	始终执行（PFS）	仅算法 5（with PFS）；算法 4 不执行
两帧认证的关键材料	群 G + gSTA / gAP	`Wrapped EAP-Initiate / EAP-Finish`；PFS 时另加群 G 与 gSTA/gAP
PMK 输入	`DHss`	`rMSK`（算法 4）或 `rMSK ‖ DHss`（算法 5）
KeyAuth	首次：数字签名；命中缓存：HMAC(ICK)	HMAC(ICK)；PFS 时追加 gSTA ‖ gAP
证书位置 / 典型失败码	关联帧的 FILS Public Key 元素 / 77、112	不发送证书 / 15、53、77、112、113

A 路径 A：FILS Public Key Authentication（无 TTP）

算法号 6 · 始终使用 Diffie-Hellman（PFS）

A1 STA → AP Authentication（Transaction Sequence = 1）

密钥建立 · 第 1 帧

发送前计算 STA

- 从 `dot11RSNAConfigDLCGroupTable` 选群 **G**（须为 AP 所支持）
- 生成临时私钥 `skSTA` 与公钥 `gSTA = scalar-op(skSTA, G)`
- 生成 16 octet `SNonce` 与随机 `FILS Session`
- 若尝试 PMKSA 缓存：RSNE 中填 `PMKID List`
- `gSTA` 按 12.4.7.2.4 编码进 `FFE`

消息字段 Authentication

Authentication Algorithm Number	**6**（FILS PK）
Auth Transaction Sequence Number	**1**
Finite Cyclic Group	G（STA 选定）
FFE	gSTA（临时 DH 公钥）
FILS Nonce	SNonce
FILS Session	本次会话标识
RSNE	1 个 Pairwise Cipher + 1 个 AKM；可选 PMKID List

注：此帧**不**携带设备证书；证书/公钥在关联帧的 FILS Public Key 元素中发送。

接收后处理 AP

1. 群 **G** 不可接受 → 回 **Status 77** 并终止
2. `gSTA` 八位串转群元素，按 NIST SP 800-56A 5.6.2.3.2 校验；失败终止
3. 提取 `SNonce`、`gSTA`
4. 检查 PMKID List：命中缓存 → 走缓存；否则 → 签名模式
5. 生成 `ANonce` 与 `(skAP, gAP)`；计算 `DHss = scalar-op(skAP, gSTA)`

A2 AP → STA Authentication（Transaction Sequence = 2）

密钥建立 · 第 2 帧

发送前计算 AP

- 设置 `Status = 0`（成功）
- 生成 16 octet `ANonce`
- 在同一群 **G** 上生成 `(skAP, gAP)`
- 回显 STA 的 `FILS Session`
- 使用缓存时填入选中的 `PMKID`
- 可延迟到发送前才计算 `DHss`

消息字段 Authentication

Authentication Algorithm Number	**6**
Auth Transaction Sequence Number	**2**
Status Code	0（或 77 等错误码）
Finite Cyclic Group	与请求相同的 G
FFE	gAP
FILS Nonce	ANonce
FILS Session	与请求相同
RSNE	协商结果；缓存时含选中 PMKID

接收后处理 STA

1. 校验 `ANA = 6`、`Group = G`、`Session` 一致；否则终止
2. `gAP` 转群元素并按 NIST 校验；失败终止
3. 提取 `ANonce`
4. 计算 `DHss = scalar-op(skSTA, gAP)`
5. 立即执行 PMK → PTK 推导，并删除 `DHss`

双方独立推导 • PMK / PMKID

```
PMK = HMAC-Hash(SNonce ‖ ANonce, DHss)
PMKID = Truncate-128(Hash(gSTA ‖ gAP))
```

PMK 长度 256 或 384 bit，由 AKM 决定；PMKID 固定 128 bit。

双方独立推导 • PTK

```
PTK = PRF-X(PMK, "FILS PTK Derivation", SPA ‖ AA ‖ SNonce ‖ ANonce)
```

SPA = STA MAC，AA = AP BSSID。非缓存 PK 的 PTK 不再附加 DHss。

拆分与使用

- **ICK** → KeyAuth 的 HMAC（缓存场景）
- **PTK-KEK** → AEAD 保护关联帧
- **TK** → 后续单播数据帧
- 生成完成后删除 `DHss`

A3 STA → AP (Re)Association Request — 密钥确认

第 3 帧 · AEAD 保护

发送前计算 STA

- **无缓存（首次 PK）**：用 STA 私钥签名
  `KeyAuth = Sig-STA(gSTA ‖ gAP ‖ SNonce ‖ ANonce ‖ STA-MAC ‖ AP-BSSID)`
- **使用缓存**：改用 HMAC
  `KeyAuth = HMAC-Hash(ICK, SNonce ‖ ANonce ‖ STA-MAC ‖ AP-BSSID ‖ gSTA ‖ gAP)`
- AAD = 四元地址 + Capability Information 起至 FILS Session（含）
- FILS Session 之后的组件用 `PTK-KEK` 做 AEAD 加密
- 把 STA 证书/公钥放入 `FILS Public Key` 元素，供 AP 验签

消息字段 (Re)Association Request

AAD（明文）	STA-MAC、AP-BSSID、SNonce、ANonce、Capability...FILS Session
FILS Session	与认证帧相同
密文部分	FILS Key Confirmation（KeyAuth）
FILS Public Key	STA 证书链 / 公钥（无缓存时必需）
RSNE	与 Auth 帧完全一致
可选	FILS HLP Container、IP Address Assignment、OCI
保护	AEAD(PTK-KEK)，AES-SIV-256/512

接收后处理 AP

1. FILS Session 不一致 → 失败
2. 用 PTK-KEK 做 AEAD 解密/验证；失败 → 失败
3. 校验 RSNE 的 AKM、cipher、RSN capabilities 与 Auth 帧一致
4. **无缓存**：从 FILS Public Key 取证书，按 RFC 5280 校验链并验签
5. **缓存**：本地构造 `Key-Auth'` 并比对
6. **失败：删除 ICK/PTK-KEK/TK/PTKSA，回 Status 112；未缓存时连 PMKSA 一并删除**
7. 成功：安装 PTKSA

A4 AP → STA (Re)Association Response — 密钥确认

第 4 帧 · AEAD 保护

发送前计算 AP

- **无缓存**：`KeyAuth = Sig-AP(gAP ‖ gSTA ‖ ANonce ‖ SNonce ‖ AP-BSSID ‖ STA-MAC)`
- **缓存**：`KeyAuth = HMAC-Hash(ICK, ANonce ‖ SNonce ‖ AP-BSSID ‖ STA-MAC ‖ gAP ‖ gSTA)`
- 构造 Key Delivery：GTK；MFP 时加 IGTK；Beacon 保护时加 BIGTK；WUR 时加 WIGTK
- AAD = AP-BSSID、STA-MAC、ANonce、SNonce、Capability...FILS Session
- 用 `PTK-KEK` 做 AEAD 加密

消息字段 (Re)Association Response

AAD（明文）	AP-BSSID、STA-MAC、ANonce、SNonce、Capability...FILS Session
FILS Session	与请求相同
密文部分	FILS Key Confirmation（KeyAuth）
Key Delivery	GTK / IGTK / BIGTK / WIGTK 及 RSC/IPN
FILS Public Key	AP 证书链 / 公钥（无缓存时）
可选	FILS HLP Container、IP 分配结果、OCI
保护	AEAD(PTK-KEK)

接收后处理 STA

1. FILS Session 一致；AEAD 解密/验证成功
2. RSNE 与 Beacon/Probe/Auth 帧一致
3. **无缓存**：用 AP 证书验签；**缓存**：构造 HMAC Key-Auth' 比对
4. 安装 TK、GTK（及 IGTK/BIGTK/WIGTK）
5. PTKSA 建立完成，立即进入数据保护

路径 A 完成 **4 帧**完成认证 + 关联 + 密钥确认；**无 TTP、无 4 次握手。**首次 PK 的 KeyAuth 为数字签名；命中 PMKSA 缓存时切换为 HMAC(ICK)。

B 路径 B：FILS Shared Key Authentication（有 TTP）—— 密钥建立 算法 4（无 PFS）/ 5（with PFS）· STA 不发送证书

B1 STA → AP Authentication（Transaction Sequence = 1）

密钥建立 · 第 1 帧

发送前计算 STA

- 生成 16 octet `SNonce` 与随机 `FILS Session`
- 决定是否尝试 PMKSA 缓存；若尝试则生成 `PMKID List`
- 若走 EAP-RP：构造 `EAP-Initiate/Reauth`（B=0、L=1、EAP ID=0、Cryptosuite≠1）
- 将其放入 `FILS Wrapped Data`
- 若 with PFS：选群 G，生成 `(skSTA, gSTA)`

消息字段 Authentication

Authentication Algorithm Number	**4**（无 PFS）或 **5**（with PFS）
Auth Transaction Sequence Number	**1**
FILS Nonce	SNonce
FILS Session	本次会话标识
RSNE	cipher / AKM；可选 PMKID List
FILS Wrapped Data	`EAP-Initiate/Reauth`（未命中缓存时）
Finite Cyclic Group	仅 PFS：G
FFE	仅 PFS：gSTA

SK 路径**不发送设备证书**；身份由 TTP 通过 EAP-ERP 验证。

接收后处理 AP

1. 含群且不支持 → 回 **Status 77**（ANA=5）并终止
2. PFS：`gSTA` 转群元素并按 NIST 校验；失败终止
3. 检查 PMKID List：命中缓存 → 用缓存 PMK，**跳过 TTP**
4. 未命中且无 EAP-Initiate → 回 **Status 53**（invalid PMKID）
5. 否则从 Wrapped Data 取出 EAP-Initiate，转发给 TTP
6. PFS：生成 `(skAP, gAP)`，可延迟到 TTP 响应后再算

B2 AP ⇄ TTP EAP-RP 往返：EAP-Initiate/Reauth → EAP-Finish/Reauth + rMSK

802.11 范围外（RADIUS/Diameter）

AP → TTP 请求

载体	RADIUS / Diameter Access-Request（示例）
EAP 报文	`EAP-Initiate/Reauth`
关键参数	B=0、L=1、EAP ID=0；Name-NAI / realm 用于定位 TTP

AP 只做解封/转发，不参与 rRK 验证。

接收后处理 TTP / AS

1. 按 RFC 6696 验证 STA 对 `rRK` 的持有
2. 验证失败 → `EAP-Finish/Reauth (R=1)`
3. 验证成功 → `EAP-Finish/Reauth (R=0)`
4. 派生本次 `rMSK`，并返回 rRK/rMSK 生命周期
5. 将 EAP-Finish 与 rMSK 一起返回 AP

TTP → AP 响应

EAP 报文	`EAP-Finish/Reauth`（R=0/1）
rMSK	成功时返回，供 AP 推导 PMK
生命周期	rRK 与 rMSK 的生命周期（L=1）

AP 保存 rMSK 后，与 STA 独立推导相同的 PMK。

B3 AP → STA Authentication（Transaction Sequence = 2）

密钥建立 · 第 2 帧

发送前计算 AP

- 生成 16 octet `ANonce`，回显 `FILS Session`
- 未用缓存：把 `EAP-Finish/Reauth` 放入 `FILS Wrapped Data`
- 使用缓存：在 `PMKID List` 中指示选中的 PMKID
- PFS：在同一群生成 `gAP`，并计算 `DHss = scalar-op(skAP, gSTA)`
- 设置 `Status = 0`（成功）

消息字段 Authentication

Authentication Algorithm Number	**4** 或 **5**（与请求一致）
Auth Transaction Sequence Number	**2**
Status Code	0；失败时 15 / 53 / 77 / 113
FILS Nonce	ANonce
FILS Session	与请求相同
FILS Wrapped Data	`EAP-Finish/Reauth`（未用缓存时）
Finite Cyclic Group	仅 PFS：G
FFE / PMKID	仅 PFS：gAP；缓存：选中 PMKID

接收后处理 STA

1. ANA 必须为 4/5；PMKID 必须匹配；FILS Session 必须相同
2. 未用缓存时必须含 PMKID 或 EAP-Finish/Reauth
3. PFS 参数必须对称：发了 `gSTA` 就必须收到 `gAP`，反之亦然
4. 处理 EAP-Finish：R=0 → 派生 `rMSK`；R=1 → 终止
5. PFS：校验 `gAP`，计算 `DHss = scalar-op(skSTA, gAP)`
6. 执行 PMK → PTK 推导；删除 `rMSK` 与 `DHss`

PMK（无 PFS / 算法 4）

`PMK = HMAC-Hash(SNonce ‖ ANonce, rMSK)`

命中 PMKSA 缓存时不重新派生 PMK，直接沿用缓存值。

PMK（with PFS / 算法 5）

`PMK = HMAC-Hash(SNonce ‖ ANonce, rMSK ‖ DHss)`

DHss 提供前向保密；PMK 生成后立即删除 rMSK 与 DHss。

PMKID 与 PTK

`PMKID = Truncate-128(Hash(EAP-Initiate/Reauth))`

`PTK = PRF-X(PMK, "FILS PTK Derivation", SPA ‖ AA ‖ SNonce ‖ ANonce)`

每次 FILS 交换都必须新建 PTKSA。

B 路径 B（续）：密钥确认与变体 KeyAuth 使用 ICK 做 HMAC；关联帧使用 PTK-KEK 做 AEAD

B4 STA → AP (Re)Association Request — 密钥确认

第 3 帧 · AEAD 保护

发送前计算 STA

- 计算 KeyAuth：`KeyAuth = HMAC-Hash(ICK, SNonce ‖ ANonce ‖ STA-MAC ‖ AP-BSSID [‖ gSTA ‖ gAP])`
- 方括号内仅在 **with PFS** 时包含 DH 公钥
- AAD = 四元地址 + Capability...FILS Session（含）
- FILS Session 之后的组件用 `PTK-KEK` 做 AEAD 加密
- 可携带 HLP Container、IP Address Assignment

消息字段 (Re)Association Request

AAD（明文）	STA-MAC、AP-BSSID、SNonce、ANonce、Capability...FILS Session
FILS Session	与认证帧相同
密文部分	FILS Key Confirmation（KeyAuth）
RSNE	与 Auth 帧完全一致
可选	FILS HLP Container、IP Address Assignment、OCI
保护	AEAD(PTK-KEK)，AES-SIV-256/512

接收后处理 AP

1. FILS Session 不一致 → 失败
2. 用 PTK-KEK 做 AEAD 解密/验证；失败 → 失败
3. 校验 RSNE 与 Auth 帧一致
4. 独立构造 `Key-Auth'`，与收到的 KeyAuth 比对
5. **失败：删除 ICK/PTK-KEK/TK/PTKSA，回 Status 112；缓存 PMKSA 可保留（可能是冒充攻击）**
6. 成功：安装 PTKSA

B5 AP → STA (Re)Association Response — 密钥确认

第 4 帧 · AEAD 保护

发送前计算 AP

- 计算 KeyAuth：`KeyAuth = HMAC-Hash(ICK, ANonce ‖ SNonce ‖ AP-BSSID ‖ STA-MAC [‖ gAP ‖ gSTA])`
- 请求/响应方向的 nonce 与 MAC 顺序相反，用于双向抗重放
- 构造 Key Delivery：GTK；MFP→IGTK；Beacon 保护→BIGTK；WUR→WIGTK
- AAD = AP-BSSID、STA-MAC、ANonce、SNonce、Capability...FILS Session
- 用 `PTK-KEK` 做 AEAD 加密

消息字段 (Re)Association Response

AAD（明文）	AP-BSSID、STA-MAC、ANonce、SNonce、Capability...FILS Session
FILS Session	与请求相同
密文部分	FILS Key Confirmation（KeyAuth）
Key Delivery	GTK / IGTK / BIGTK / WIGTK 及 RSC/IPN
可选	FILS HLP Container、IP 分配结果、OCI
保护	AEAD(PTK-KEK)

接收后处理 STA

1. FILS Session 一致；AEAD 解密/验证成功
2. RSNE 与 Beacon/Probe/Auth 帧一致
3. 构造 `Key-Auth'` 与收到的 KeyAuth 比对
4. 安装 TK、GTK（及 IGTK/BIGTK/WIGTK）
5. PTKSA 建立完成，立即进入数据保护

路径 B 完成 **4 帧**完成认证 + 关联 + 密钥确认；TTP 交互只发生在 AP 与 TTP 之间，**不增加 STA 的往返次数。**

4 路径变体与失败分支 同一套四帧框架，不同字段与密钥输入

SK 无 PFS（算法 4）

- 认证帧**无** Finite Cyclic Group / FFE
- `PMK = HMAC(SNonce‖ANonce, rMSK)`
- KeyAuth 不含 gSTA/gAP
- 依赖 rMSK 的保密性，无前向保密

SK with PFS（算法 5）

- 两帧认证都带群 G 与 FFE
- `PMK = HMAC(SNonce‖ANonce, rMSK‖DHss)`
- KeyAuth 追加 gSTA‖gAP
- 提供前向保密；群不支持 → 77

PMKSA 缓存（SK / PK）

- Auth1 的 RSNE 带 PMKID List；Auth2 指示选中的 PMKID
- 命中时**不联系 TTP**、不新建 PMKSA，沿用缓存 PMK/PMKID
- 仍必须新建 PTKSA
- 2024：缓存 + DH 时，PTK 推导追加 `DHss`
- PK 缓存时 KeyAuth 由签名改为 HMAC(ICK)

失败与状态码

码	含义
0	成功
15	TTP 挑战失败（EAP-RP 验证失败）
53	无效 PMKID（未命中缓存且无 EAP-Initiate）
77	有限循环群不支持
112	FILS 认证失败（KeyAuth / AEAD 等）
113	AP 不认识 STA 所指的认证服务器

密钥删除规则

- PMK/PMKID 生成后：不可恢复地删除 rMSK 与 DHss
- PTK 生成后：删除 DHss
- AP 密钥确认失败：删 ICK/PTK-KEK/TK/PTKSA；未缓存则删 PMKSA
- STA 密钥确认失败：删 ICK/PTK-KEK/PMK/TK，放弃交换
- 认证成功：删除密钥建立阶段的非持久秘密材料

KeyAuth 四种构造速查

场景	请求方向（STA → AP）	响应方向（AP → STA）
SK 无 PFS（4）	`HMAC-Hash(ICK, SNonce‖ANonce‖STA-MAC‖AP-BSSID)`	`HMAC-Hash(ICK, ANonce‖SNonce‖AP-BSSID‖STA-MAC)`
SK with PFS（5）	`HMAC-Hash(ICK, SNonce‖ANonce‖STA-MAC‖AP-BSSID‖gSTA‖gAP)`	`HMAC-Hash(ICK, ANonce‖SNonce‖AP-BSSID‖STA-MAC‖gAP‖gSTA)`
PK + 缓存	`HMAC-Hash(ICK, SNonce‖ANonce‖STA-MAC‖AP-BSSID‖gSTA‖gAP)`	`HMAC-Hash(ICK, ANonce‖SNonce‖AP-BSSID‖STA-MAC‖gAP‖gSTA)`
PK 无缓存	`Sig-STA(gSTA‖gAP‖SNonce‖ANonce‖STA-MAC‖AP-BSSID)`	`Sig-AP(gAP‖gSTA‖ANonce‖SNonce‖AP-BSSID‖STA-MAC)`

5 密钥推导：从 rRK / DHss 到 PTKSA

PMK 由两条路径之一产生；PTK 每次都重新推导

输入材料

rRK	EAP-RP	rMSK	skSTA / skAP	DHss
SK：STA ↔ TTP 长期共享	EAP-Initiate / EAP-Finish	SK：TTP → AP / STA	PK / SK+PFS 临时私钥	scalar-op(私钥, 对端公钥)

```
PMK = HMAC-Hash( SNonce ‖ ANonce ,  rMSK [‖ DHss]  /  DHss )
PMKID = Truncate-128( Hash(EAP-Initiate/Reauth) )  ·  或  Truncate-128( Hash(gSTA ‖ gAP) )
```

▼ PMK 256 / 384 bit，由 AKM 决定

```
PTK = PRF-X( PMK , "FILS PTK Derivation" , SPA ‖ AA ‖ SNonce ‖ ANonce [‖ DHss] )
X = 512/768/896/1280 + TK_bits；方括号仅在 2024 的“PMKSA 缓存 + DH”场景加入
```

▼

ICK	PTK-KEK	TK	FILS-FT / KDK
KeyAuth 的 HMAC 完整性密钥	AEAD 保护关联帧 / EAPOL-Key	单播数据帧加密	FT 初始移动域 / WUR、secure HE-LTF

各路径的 PMK / PMKID

路径	PMK	PMKID
SK 无 PFS（4）	HMAC-Hash(SNonce‖ANonce, rMSK)	Truncate-128(Hash(EAP-Initiate/Reauth))
SK with PFS（5）	HMAC-Hash(SNonce‖ANonce, rMSK‖DHss)	
PK（6）	HMAC-Hash(SNonce‖ANonce, DHss)	Truncate-128(Hash(gSTA‖gAP))
PMKSA 缓存	沿用缓存 PMK	沿用原 PMKID

Hash 由协商 AKM 决定（SHA-256 / SHA-384）。PMK 生成后删除 rMSK 与 DHss。

PTK 各字段长度与 KeyAuth

AKM	ICK	PTK-KEK	FILS-FT
00-0F-AC:14	256	256	—
00-0F-AC:15	384	512	—
00-0F-AC:16	256	256	256
00-0F-AC:17	384	512	384

KeyAuth 请求方向：HMAC-Hash(ICK, SNonce‖ANonce‖STA-MAC‖AP-BSSID[‖gSTA‖gAP])
响应方向：HMAC-Hash(ICK, ANonce‖SNonce‖AP-BSSID‖STA-MAC[‖gAP‖gSTA])
PK 首次认证（无缓存）：改为 Sig-STA / Sig-AP 数字签名。

口径说明（以 IEEE 802.11-2024 §12.11 原文为准）

1. **算法号**：FILS SK 无 PFS = **4**，SK with PFS = **5**，PK = **6**（§9.4.1.1）。前文中的“ANA-8”并非 FILS 分配值。
2. **完整性密钥名称**：FILS 使用 **ICK**（Integrity Check Key）计算 KeyAuth；部分资料写作 KCK。PTK 中对应字段为 ICK / PTK-KEK / TK。
3. **PMK 推导**：标准公式为 **HMAC-Hash(SNonce||ANonce, ...)**（§12.11.2.5.2），不是 KDF-384 的 “FILS PMKSA Derivation” 形式；PMKID 公式见上表。
4. **PK 路径的证书位置**：Auth 帧只携带 **FFE = gSTA/gAP**；设备证书/公钥在**关联帧**的 FILS Public Key 元素中传输。首次 PK 的 KeyAuth 是**数字签名**，只有命中 PMKSA 缓存时才用 HMAC(ICK)。
5. **TTP 与 AS**：标准正文多称 Authentication Server（AS）；TTP 是同一信任实体的架构称谓。AP↔TTP 的传输（RADIUS/Diameter）不在 802.11 范围内。

快速性来源 ①
认证帧本身承载密钥建立材料（DH 公钥或 Wrapped EAP-RP），两帧认证后即可产生 PMK。

快速性来源 ②
密钥确认直接嵌入 (Re)Association Request/Response，不再单独运行 4 次 EAPOL 握手。

快速性来源 ③
SK 的 TTP 交互发生在 AP↔TTP，不增加 STA 空中往返；PK 路径完全去除 TTP。

四帧完成后的状态

帧	完成后的双方状态
① Auth (Seq=1)	STA 发出 SNonce、gSTA（PK/PFS）、FILS Session；AP 完成群/公钥校验并生成 ANonce、gAP、DHss（如适用）
② Auth (Seq=2)	双方得到对端 nonce 与 DH 公钥，独立推导 PMK / PMKID / PTK（ICK、PTK-KEK、TK），并删除 rMSK / DHss
③ (Re)Assoc Request	AP 用 PTK-KEK 解密并验证 KeyAuth，确认双方密钥一致；AP 侧安装 PTKSA，准备 Key Delivery
④ (Re)Assoc Response	STA 验证 AP 的 KeyAuth，安装 PTKSA 与 GTK（及 IGTK/BIGTK/WIGTK）；FILS 完成，可立即收发受保护数据